

Министерство образования Республики Беларусь

Учреждение образования
«Белорусский государственный университет информатики и
радиоэлектроники»

Кафедра ЭВМ

Лабораторная работа № 3
«Тактирование. Питание. Энергопотребление»
Вариант №9

Выполнил:
Студент группы 250501 Снитко Д.А.

Проверил:
ассистент каф. ЭВМ _____ Шеменков В.В.

Минск 2025

1. Цель работы

Цель: изучить принципы построения систем с низким энергопотреблением на базе микроконтроллера MSP430F5529.

Задача: написать программу, переключающую режимы тактирования, питания и энергопотребления в соответствии с заданием.

2. Исходные данные

Для выполнения работы используется плата MSP-EXP430F5529 и интегрированная среда разработки Code Composer Studio.

Необходимо выполнить задание варианта № 9.

LPM	Vcore, В	Частота
3	1,9 — 1,4	ACLK — 0,5 МГц / 1 МГц

3. Выполнение работы

3.1. Изменение частоты питания и энергопотребления

Для выполнения лабораторной работы следовало измерить напряжение, ток и частоту в разных режимах.

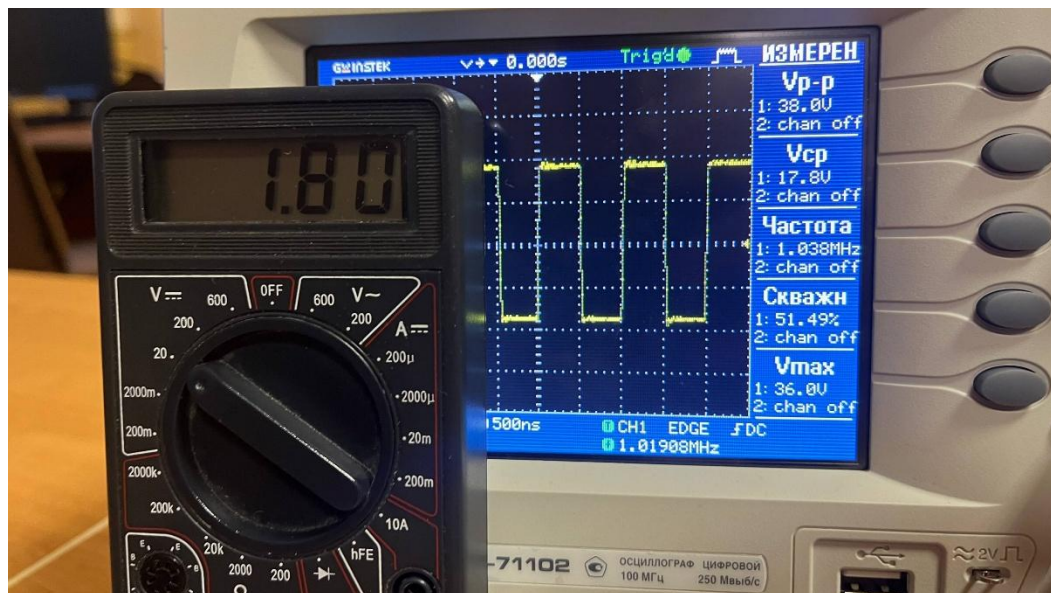


Рисунок 3.1.1 – Напряжение в обычном режиме



Рисунок 3.1.2 – Сила тока в режиме ACLK – 0,5 МГц



Рисунок 3.1.3 – Сила тока в режиме ACLK – 1 МГц



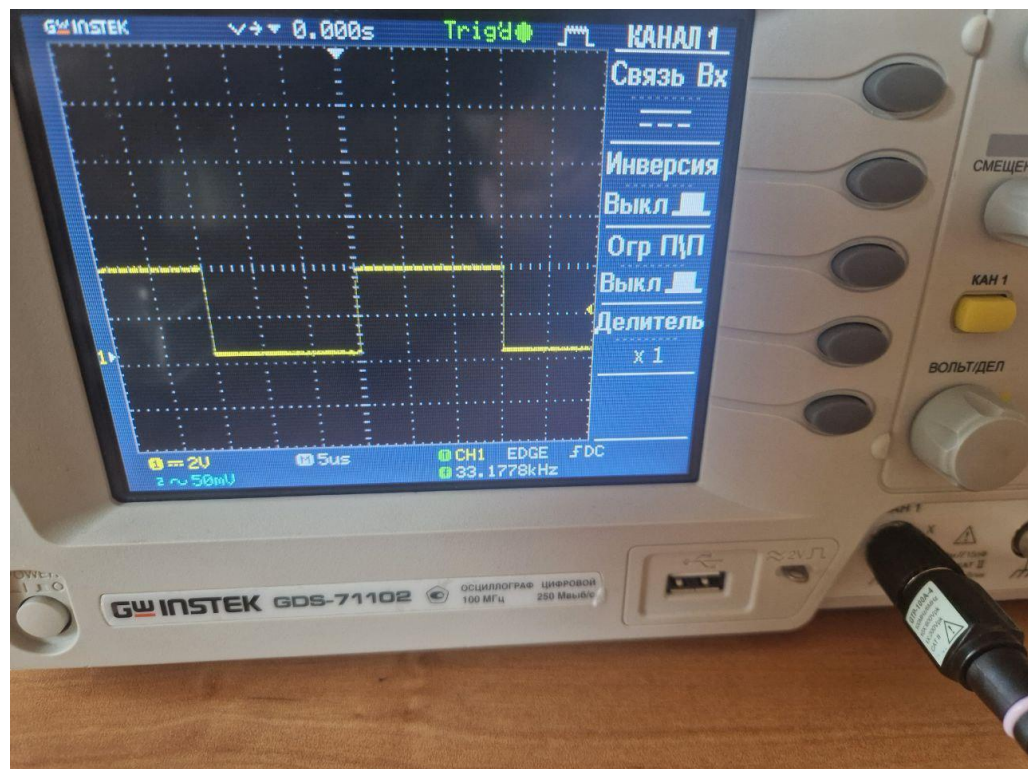
Рисунок 3.1.4 – Напряжение в режиме пониженного энергопотребления.



Рисунок 3.1.5 – Сила тока в режиме ACLK – 0,5 МГц



Рисунок 3.1.6 – Сила тока в режиме ACLK – 1 МГц



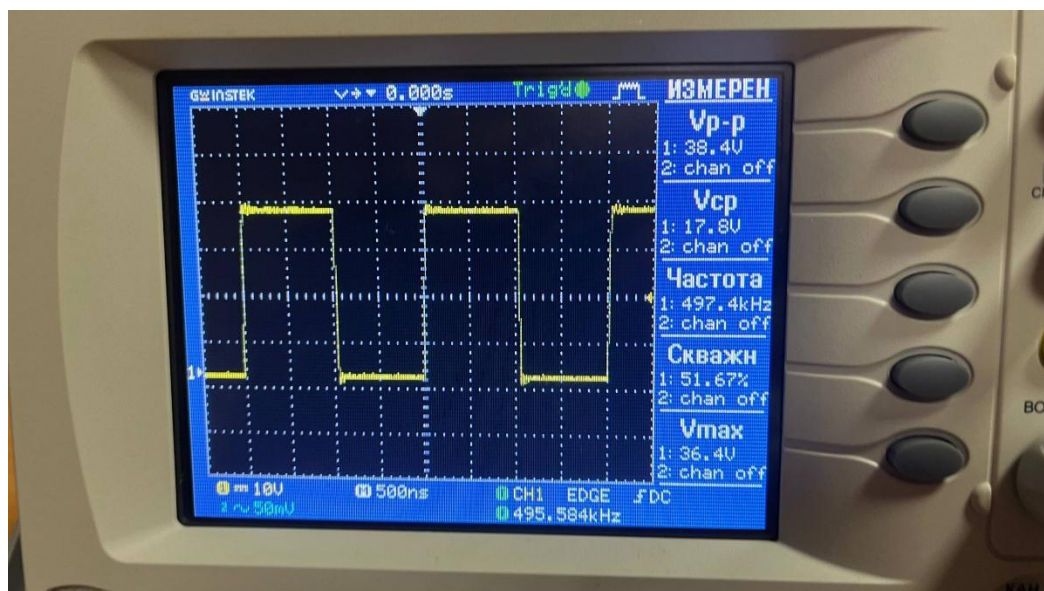


Рисунок 3.1.7 – Частота ACLK – 0,5 МГц

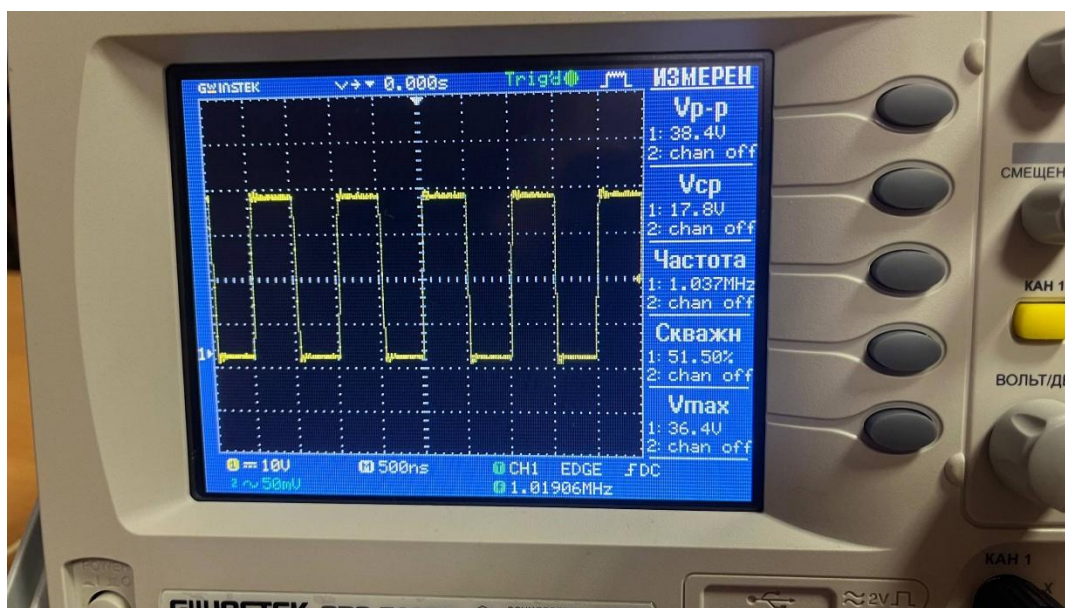


Рисунок 3.1.8 – Частота ACLK – 1 МГц

3.2. Код программы

```
#include <msp430f5529.h>

// Определения светодиодов и кнопок
#define LED1    BIT0    // P1.0 – индикация активного режима
#define LED3    BIT7    // P4.7 – индикация LPM3
#define LED4    BIT1    // P1.1 – высокая частота (1 МГц)
#define LED5    BIT3    // P1.3 – низкая частота (500 кГц)
```

```

#define LED8      BIT2      // P1.2 – мигание от таймера TA0

#define S1        BIT7      // P1.7 – переключение частоты DCO
(500 кГц ↔ 1 МГц)
#define S2        BIT2      // P2.2 – вход/выход в LPM3

volatile unsigned int freq_mode = 0;    // 0 – 500кГц, 1 –
1МГц
volatile unsigned int sleep_mode = 0;   // 0 – активный, 1 –
LPM3
volatile unsigned int vcore_level = 0;  // Уровень Vcore: 0
(~1.4 В) или 2 (~1.9 В)
volatile unsigned char debounce_s1 = 0; // Флаг подавления
дребезга для S1
volatile unsigned char debounce_s2 = 0; // Флаг подавления
дребезга для S2

// Настройка DCO на ~500 кГц
void initClockDCO_500kHz() {
    UCSCTL3 = SELREF__REFOCLK | FLLREFDIV_0; // REFOCLK как
источник FLL
    UCSCTL4 = SELM__DCOCLK | SELS__DCOCLK | SELA__REFOCLK; //
MCLK, SMCLK = DCOCLK, ACLK = REFOCLK
    __bis_SR_register(SCG0); // Отключить FLL для настройки
    UCSCTL0 = 0x0000;        // Минимальные DCOх, MODх
    UCSCTL1 = DCORSEL_1;     // Диапазон DCO ~0.3-1.5 МГц
    UCSCTL2 = FLLD__1 | 14;  // FLLD=1, N=14 (15*32768 ≈ 0.491
МГц)
    __bic_SR_register(SCG0); // Включить FLL
    __delay_cycles(500000); // Задержка стабилизации
    do {
        UCSCTL7 &= ~(XT2OFFG + XT1LFOFFG + DCOFFG); // Сброс
флагов ошибок
        SFRIFG1 &= ~OFIFG;
    } while (SFRIFG1 & OFIFG); // Ожидание стабилизации DCO
}

// Настройка DCO на ~1 МГц
void initClockDCO_1MHz() {
    UCSCTL3 = SELREF__REFOCLK | FLLREFDIV_0; // REFOCLK как
источник FLL
    UCSCTL4 = SELM__DCOCLK | SELS__DCOCLK | SELA__REFOCLK; //
MCLK, SMCLK = DCOCLK, ACLK = REFOCLK
    __bis_SR_register(SCG0); // Отключить FLL для настройки

```

```

UCSCTL0 = 0x0000;          // Минимальные DCOх, MODх
UCSCTL1 = DCORSEL_2;       // Диапазон DCO ~1-2 МГц
UCSCTL2 = FLLD__1 | 30;    // FLLD=1, N=30 (31*32768 ≈ 1.015
МГц)

__bic_SR_register(SCG0); // Включить FLL
__delay_cycles(1000000); // Задержка стабилизации
do {
    UCSCTL7 &= ~(XT2OFFG + XT1LFOFFG + DCOFFG); // Сброс
флагов ошибок
    SFRIFG1 &= ~OFIFG;
} while (SFRIFG1 & OFIFG); // Ожидание стабилизации DCO
}

// Пошаговое изменение Vcore (0 ↔ 2)
void setVcoreLevel(unsigned int new_level) {
    PMMCTL0_H = PMMPW_H; // Открыть PMM (пароль)
    SVSMHCTL = SVSHE | (SVSHRVL0 * new_level) | SVMHE |
(SVSMHRRLO * new_level); // SVS/SVM high
    SVSMLCTL = SVSLE | SVMLE | (SVSMLRRL0 * new_level); //
SVS/SVM low
    while (!(PMMIFG & SVSMLDLYIFG)); // Ожидание стабилизации
задержки
    PMMIFG &= ~(SVMLVLRIFG + SVMLIFG); // Сброс флагов
    PMMCTL0_L = PMMCOREV0 * new_level; // Установить новый
уровень Vcore
    if (PMMIFG & SVMLIFG) while (!(PMMIFG & SVMLVLRIFG)); //
Ожидание стабилизации уровня
    SVSMLCTL = SVSLE | (SVSLRVL0 * new_level) | SVMLE |
(SVSMLRRL0 * new_level); // Обновить SVS/SVM low
    PMMCTL0_H = 0; // Закрыть PMM
}

// Настройка портов ввода-вывода
void initGPIO() {
    // Светодиоды как выходы
    P1DIR |= LED1 | LED4 | LED5 | LED8; // P1.0, P1.1, P1.3,
P1.2
    P4DIR |= LED3; // P4.7
    P7DIR |= BIT7; // P7.7 — вывод MCLK
    P4DIR |= BIT0; // P4.0 — вывод SMCLK
    P1SEL &= ~(LED1 | LED4 | LED5 | LED8); // Отключить
альтернативные функции для светодиодов
    P4SEL &= ~LED3;

```



```

        P7SEL |= BIT7;                // Включить MCLK на
P7.7
        P4SEL |= BIT0;                // Включить SMCLK на
P4.0
        P1OUT |= LED1;                // Включить LED1
(активный)
        P1OUT |= LED5;                // Включить LED5 (500
кГц по умолчанию)
        P1OUT &= ~(LED4 | LED8);      // LED4, LED8 выключены
        P4OUT &= ~LED3;                // LED3 выключен

// Кнопка S1 (P1.7)
P1DIR &= ~S1;        // Вход
P1REN |= S1;         // Подтяжка включена
P1OUT |= S1;         // Подтяжка к Vcc
P1IE |= S1;          // Разрешить прерывание
P1IES |= S1;         // Прерывание по спаду
P1IFG &= ~S1;        // Сброс флага

// Кнопка S2 (P2.2)
P2DIR &= ~S2;
P2REN |= S2;
P2OUT |= S2;
P2IE |= S2;
P2IES |= S2;
P2IFG &= ~S2;
}

// Настройка таймера TA0 для мигания LED8 (работает от ACLK
для совместимости с LPM3)
void initTimer() {
    TA0CCR0 = 16384;        // Период: ~1 Гц при ACLK 32.768
кГц (32768/2 = мигание 1 раз в сек)
    TA0CCTL0 = CCIE;        // Включить прерывание
    TA0CTL = TASSEL__ACLK | MC__UP | TACLK; // ACLK, режим
ур, сброс
}

// Настройка WDT для подавления дребезга (~32 мс)
void initWDT() {
    WDTCTL = WDTPW | WDTTMSSEL | WDTSSSEL__ACLK | WDTIS_4; //
ACLK/2^15 (~32 мс)
    SFRID1 |= WDTIE; // Разрешить прерывания WDT
}

```

```

// Обработчик WDT для подавлениядребезга
#pragma vector = WDT_VECTOR
__interrupt void WDT_ISR(void) {
    if (debounce_s1) {
        debounce_s1++;
        if (debounce_s1 >= 2) { // ~32 мс
            P1IE |= S1; // Восстановить прерывание S1
            debounce_s1 = 0;
        }
    }
    if (debounce_s2) {
        debounce_s2++;
        if (debounce_s2 >= 2) { // ~32 мс
            P2IE |= S2; // Восстановить прерывание S2
            debounce_s2 = 0;
        }
    }
}

// Обработчик S1: переключение частоты DCO и Vcore по спаду
#pragma vector = PORT1_VECTOR
__interrupt void Port_1(void) {
    if (P1IFG & S1) {
        if (!(P1IN & S1) && !debounce_s1) { // Проверка:
кнопка нажата (спад) и нетдребезга
            debounce_s1 = 1; // Установить флаг подавления
дребезга
            P1IE &= ~S1; // Отключить прерывание на время
debounce
            freq_mode ^= 1; // Переключить режим (500 кГц ↔
1 МГц)

            if (freq_mode == 0) { // 500 кГц, Vcore=0
                // Сначала снижаем частоту, потом Vcore
                initClockDCO_500kHz();
                __delay_cycles(1000); // Задержка для
стабилизации

                if (vcore_level == 2) {
                    setVcoreLevel(1); // Пошагово: 2→1
                    setVcoreLevel(0); // 1→0
                    vcore_level = 0;
                }
                P1OUT &= ~LED4; // LED4 off

```

```

        P1OUT |= LED5; // LED5 on (500 кГц)
    } else { // 1 МГц, Vcore=2
        // Сначала поднимаем Vcore, потом частоту
        if (vcore_level == 0) {
            setVcoreLevel(1); // Пошагово: 0→1
            setVcoreLevel(2); // 1→2
            vcore_level = 2;
        }
        initClockDCO_1MHz();
        __delay_cycles(1000); // Задержка для
стабилизации

        P1OUT |= LED4; // LED4 on (1 МГц)
        P1OUT &= ~LED5; // LED5 off
    }
}

P1IFG &= ~S1; // Сброс флага прерывания
}

// Обработчик S2: вход/выход в LPM3 по спаду
#pragma vector = PORT2_VECTOR
__interrupt void Port_2(void) {
    if (P2IFG & S2) {
        if (!(P2IN & S2) && !debounce_s2) { // Проверка:
кнопка нажата (спад) и нетдребезга
            debounce_s2 = 1; // Установить флаг подавления
дребезга

            P2IE &= ~S2; // Отключить прерывание на время
debounce

            sleep_mode ^= 1; // Переключить режим (активный
↔ LPM3)

            if (sleep_mode) {
                P1OUT &= ~LED1; // LED1 off (LPM3)
                P4OUT |= LED3; // LED3 on (LPM3)
                __bis_SR_register_on_exit(LPM3_bits); //
Вход в LPM3
            } else {
                P4OUT &= ~LED3; // LED3 off
                P1OUT |= LED1; // LED1 on (активный)
                __bic_SR_register_on_exit(LPM3_bits); //
Выход из LPM3
            }
        }
    }
}

```

```

        P2IFG &= ~S2; // Сброс флага прерывания
    }
}

// Обработчик таймера TA0: мигание LED8
#pragma vector = TIMER0_A0_VECTOR
__interrupt void Timer_A(void) {
    P1OUT ^= LED8; // Переключение LED8
}

int main(void) {
    WDTCTL = WDTPW | WDTHOLD; // Отключить WDT
    initGPIO();                // Инициализация портов
    initClockDCO_500kHz();     // Инициализация DCO (~500 кГц)
    initTimer();               // Инициализация таймера TA0
    initWDT();                 // Инициализация WDT для debounce
    __enable_interrupt();      // Разрешить прерывания

    while (1) {
        __no_operation();      // Ожидание прерываний для
экономии энергии
    }
}

```

4. Выводы

В ходе выполнения лабораторной работы удалось изучить принципы построения систем с низким энергопотреблением на базе микроконтроллера MSP430F5529. Удалось написать программу, переключающую режимы тактирования, питания и энергопотребления в соответствии с заданием.